0. **Code:** karuturim2445 karuturi mid2 A.zip

1. mid2 A/Program.cs
2. mid2 b/Program.cs

1. **mid2 A/Program.cs**

```
using System.Diagnostics;
using System.Transactions;

namespace mid_2
{
    internal class A
    {
        static void Main(string[] args)
        {
            Console.WriteLine("enter n value");

            int n = int.Parse(Console.ReadLine());

            int[] arr = new int[n];


            for (int i = 0; i < n; i++)
            {
                arr[i] = int.Parse(Console.ReadLine());

            }

            double result = runavg(arr);

            Console.WriteLine(result);



        }
        static double runavg(int[] arr)
        {
```

just hard code the values in your array.

A1

1. **mid2 A/Program.cs**

```
            int count = 0;
            int prevsum = arr[0];
            int runtotal = 0;

            for (int i = 1; i < arr.Length; i++)
            {


                if (arr[i] > prevsum / i)
                {
                    runtotal += arr[i];
                    count++;
                }
                prevsum += arr[i];

            }

            if (count > 0)
            {
                return (double) runtotal / count;

            }
            return 0;



        }
    }
}
```

A2b

A2

A2c

A3

2. mid2 b/Program.cs

```
namespace mid2_b
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int[] arr = {
                7,
                3,
                1,
                8,
                2,
                4,
                5,
                0,
                9
            };

            int[] arr2 = loop(arr);

            Console.WriteLine(string.Join(",", arr2));

        }

        static int[] loop(int[] arr)
        {

            int len = (arr.Length * (arr.Length + 1)) / 2;
            int[] newarr = new int[len];
            int index = arr.Length;

```

B1

B2

2. **mid2 b/Program.cs**

```
        int n = 0;
        int small = int.MaxValue;
        int lowindex = newarr[0];
        int highindex = arr.Length;

        for (int i = 0; i < arr.Length; i++)
        {
            newarr[n] = arr[i];
            n++;
        }

        for (int i = lowindex; i < highindex; i++)
        {

            if (newarr[i] < small)

            {
                small = newarr[i];

            }

            for (int k = highindex; k < newarr.Length; k++)
            {
                for (int j = highindex; j < newarr.Length; j++)
                {
                    if ((newarr[k] != small))
                    {
                        newarr[highindex - 1] = newarr[lowindex];
```

B2

could use i.

should go for 0 → arr.length

B4

need to find the smallest number from the block.

For each small value?

k is next block

What's j k?

remove it?

not useful.

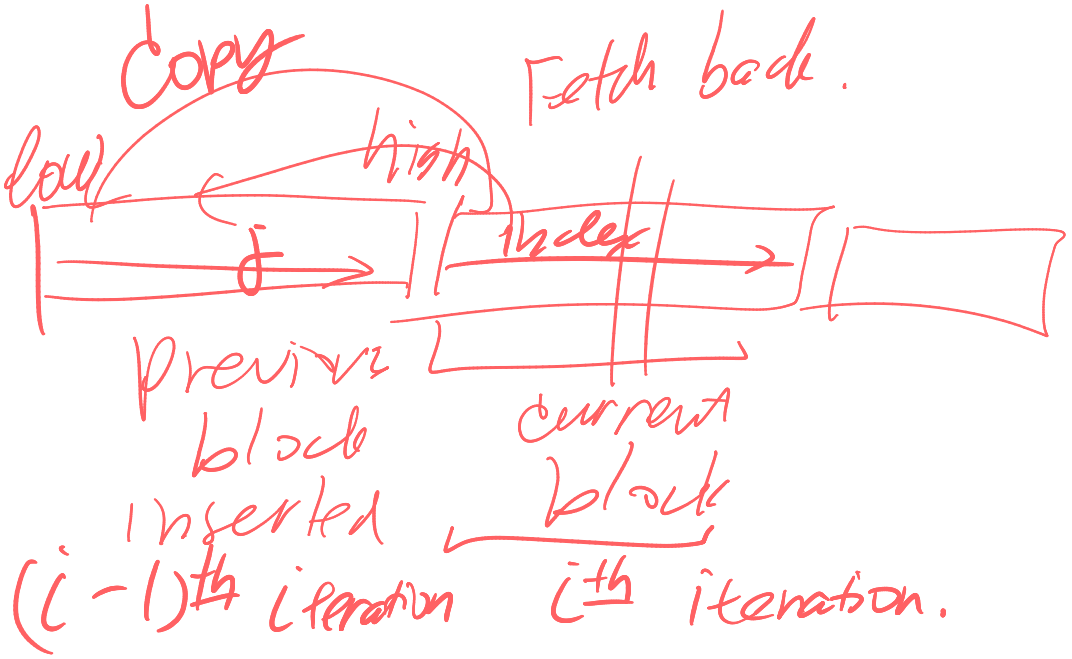
B3 ?

why high index?

2. mid2 b/Program.cs

```
                    }

                }


            }
            highindex++;


        }


        return newarr;


    }
  }
}
```

remove min.

copy

Fetch back.